



CMPT 295

Unit - Machine-Level Programming

Lecture 20 – Assembly language –

Function Call and Stack - Passing Data – Part 2

Today's Menu

- Introduction
 - C program -> assembly code -> machine level
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – **Stack** – Recursion
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions (cont'd)
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point operations

Overview of Function Call

What happens when a function (**caller**) calls another function (**callee**) in **x86-64 Assembly Language**?

1. **Control** (execution flow)
2. **Data**
3. **Memory** -> stack memory segment

Looking into the **function call conventions** of **x86-64 Assembly Language**

8

2. **Passing data** mechanism – using **stack** **x86-64** function call convention

1. **Caller** and **callee** functions must obey **function call convention** (protocol) when passing data during function call

➤ **Caller:**

- Before calling the **callee** function, the **caller** must ensure that the **callee's arguments** (1 to 6) are loaded into their specific registers:

If there is a ... **1st argument**, then it is moved to **%rdi** (or **%edi**, or **%di** or **%dil**)

If there is a ... **2nd argument**, then it is moved to **%rsi** (or **%esi**, or **%si** or **%sil**)

If there is a ... **3rd argument**, then it is moved to **%rdx** (or **%edx**, or **%dx** or **%dl**)

If there is a ... **4th argument**, then it is moved to **%rcx** (or **%ecx**, or **%cx** or **%cl**)

If there is a ... **5th argument**, then it is moved to **%r8** (or **%r8d**, or **%r8w** or **%r8b**)

If there is a ... **6th argument**, then it is moved to **%r9** (or **%r9d**, or **%r9w** or **%r9b**)

➤ **Callee:**

- Before returning to **caller**, **callee** must ensure that the **returned value** is loaded into its specific register, i.e., **%rax** (or **%eax**, or **%ax** or **%al**)

2. **Passing data** mechanism – using **stack** **x86-64** function call convention

1. **Caller** and **callee** functions must obey **function call convention** (protocol) when passing data during function call

➤ **Caller:**

➤ Before calling the **callee** function, the **caller** must ensure that the **callee**'s arguments (1 to 6) are loaded into their specific registers:

➤ What if the **callee** function has more than **6 arguments**?

➤ If a **callee** function has more than 6 arguments ... then the **caller** must push the rest of the **arguments** on the **stack** in reverse order before calling the **callee** function

➤ **Callee:**

➤ Before returning to **caller**, **callee** must ensure that the returned value is loaded into its specific register, i.e., **%rax** (or **%eax**, or **%ax** or **%al**)

2. *Passing data* mechanism – using **stack** **x86-64** function call convention

2. When a function (**caller**) is passing data that is the **memory address** of one of its local variables (i.e., a pointer in C) during function call – when calling the **callee**
 - The **caller** must make use of the **stack** in order to create such **memory address**

Passing data mechanism – Examples of local variables, arguments (> 6) and pointers on the stack

```
long call_proc()
{
    long x1 = 1;
    int x2 = 2;
    short x3 = 3;
    char x4 = 4;
    proc(x1, &x1, x2, &x2,
        x3, &x3, x4, &x4);

    return (x1+x2) * (x3-x4);
}
```

How to push
x4 and &x4
onto stack?

```
call_proc:
    subq    $40, %rsp
    movq    $1, 32(%rsp)
    movl    $2, 28(%rsp)
    movw    $3, 26(%rsp)
    movb    $4, 25(%rsp)
    movq    32(%rsp), %rdi
    movl    28(%rsp), %edx
    leaq    25(%rsp), %rax
    movq    %rax, 8(%rsp)
    movl    $4, (%rsp)
    leaq    32(%rsp), %rsi
    leaq    28(%rsp), %rcx
    leaq    26(%rsp), %r9
    movl    $3, %r8d
    call    proc
    ...
```

local
variables

Passing data mechanism – Example – cont'd

```
void proc(long a1, long *alp,  
          int a2, int *a2p,  
          short a3, short *a3p,  
          char a4, char *a4p)  
{  
  
    *alp += a1;  
    *a2p += a2;  
    *a3p += a3;  
    *a4p += a4;  
  
    return;  
}
```

```
proc:  
    movb    8(%rsp), %r10b  
    movq    16(%rsp), %rax  
    addq    %rdi, (%rsi)  
    addl    %edx, (%rcx)  
    addw    %r8w, (%r9)  
    addb    %r10b, (%rax)  
    ret
```

Passing data mechanism – Example – cont'd

```
long call_proc()  
{  
    long x1 =1;  
    int x2 =2;  
    short x3 = 3;  
    char x4 = 4;  
    proc(x1, &x1, x2, &x2,  
        x3, &x3, x4, &x4);  
  
    return (x1+x2)*(x3-x4);  
}
```

How to push
x4 and &x4
onto stack?

```
call_proc:  
    ...  
    call    proc  
    movslq  28(%rsp), %rcx  
    addq    32(%rsp), %rcx  
    movswq  26(%rsp), %rax  
    movsbq  25(%rsp), %rdx  
    subq    %rdx, %rax  
    imulq   %rcx, %rax  
    addq    $40, %rsp  
    ret
```


gcc and us
(software developers)
must abide to ...

Summary

➔ **Passing data** mechanism - **x86-64 function call convention:** ^{protocol}

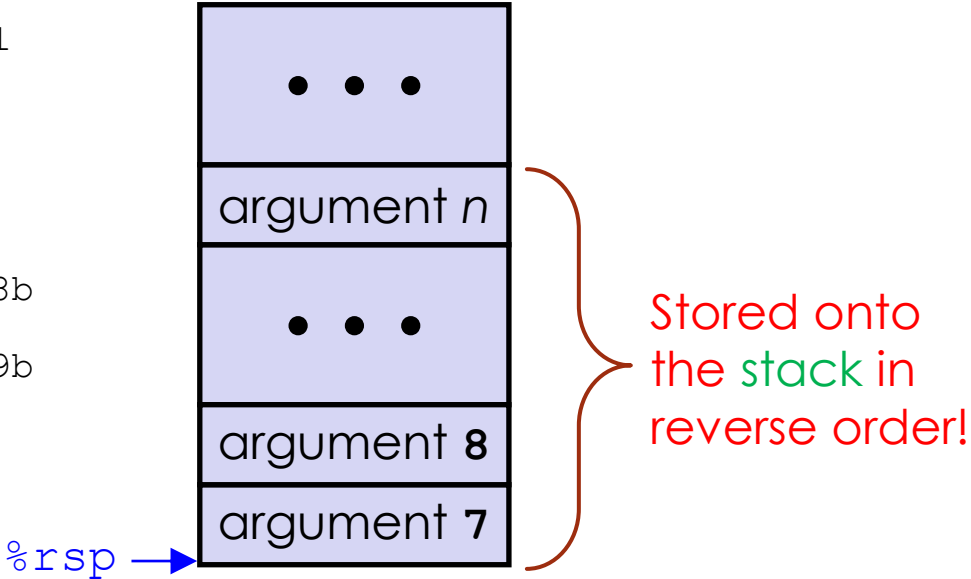
First 6 arguments

argument 1 ->	%rdi	or %edi, or %di or %dil
argument 2 ->	%rsi	or %esi, or %si or %sil
argument 3 ->	%rdx	or %edx, or %dx or %dl
argument 4 ->	%rcx	or %ecx, or %cx or %cl
argument 5 ->	%r8	or %r8d, or %r8w or %r8b
argument 6 ->	%r9	or %r9d, or %r9w or %r9b

return value

%rax

Remaining arguments
Stack



- ➔ If an arguments is a **pointer**, **caller** must make use of the **stack** in order to create such memory address before storing this memory address either in the appropriate register (if < 7th argument) or on the stack (if >= 7th argument)

Next Lecture

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, move operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – **Stack** – Recursion
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point operations